UpCharika: Multilingual Clinical AI Assistant

Final Year Engineering Project Study Guide & Comprehensive Technical Manual

Domain: Healthcare AI / NLP / RAG	**Target Languages:** English, Marathi, Hindi
Core Framework: LangChain + FAISS + Groq LPUs	**UI Engine:** Streamlit + Custom CSS SaaS
Vector Index: 119 MB FAISS (Medical Textbooks)	**Voice Synthesis:** Latency-Optimized gTTS

1. Executive Summary & Problem Statement

Modern Large Language Models (LLMs) such as ChatGPT or Claude demonstrate remarkable conversational fluency, yet they pose critical hazards when applied directly to clinical triage. Standard models frequently exhibit **hallucination**—the fabrication of plausible-sounding but medically inaccurate diagnoses and dosages. Furthermore, over 70% of semi-urban and rural patients across Maharashtra communicate primarily in regional dialects (Marathi and Hindi) and encounter steep language barriers when seeking medical guidance online.

UpCharika addresses these clinical and accessibility gaps by implementing a **True Retrieval-Augmented Generation (RAG)** architecture. Rather than permitting the AI to guess, UpCharika enforces mathematical similarity search across a 119 MB vectorized library of peer-reviewed clinical textbooks (FAISS), grounds the AI prompt with verifiable textbook excerpts, and produces structured 5-part triage guidance with low-latency spoken voice consultation in English, Marathi, and Hindi.

2. What Makes UpCharika Unique (The 5 Key Differentiators)

The table below contrasts standard chatbots against UpCharika across essential clinical evaluation metrics:

Evaluation Dimension	Generic Chatbots (ChatGPT / Basic Clones)	UpCharika (Your Project)
Medical Accuracy	Generates advice purely from internal probabilistic weights; susceptible to hallucination.	**Strict RAG Grounding:** Extracts verified paragraphs from a 119 MB textbook vectorstore before generating advice.
Language Accessibility	Literal machine translation; frequently mistranslates clinical terms in Devanagari script.	**Native Devanagari Triage:** Dedicated Marathi (■■■■■) and Hindi (■■■■■■) triage cards and multi-tier translation fallback.
Spoken Voice Latency	Text only, or synthesizes entire documents causing an unacceptable 60–90s freeze.	**Optimized Clinical Voice (2.5s):** extract_voice_summary extracts the primary 2 actionable sentences for instant audio.

Clinical Structure	Unstructured wall of text; patient struggles to locate urgent emergency warnings.	**Standardized 5-Section Protocol:** Cause, Solution, Medicine, When to See Doctor, and Emergency Red Flags.
Offline Resilience & Viva Proof	Fails completely if cloud API expires or network connection drops.	**Test Mode & Direct FAISS Extraction:** Functions even without Groq, verifiable via the purple 'Direct Textbook' badge.

3. End-to-End System Architecture & Data Flow

When an authenticated patient inputs an inquiry (e.g. *'What is jaundice and what diet should I eat?'*), the system executes the following 6 sequential steps:

- **Step 1: Session & Authentication Gate:** medibot.py verifies user session authenticity using salted bcrypt hashes and EncryptedCookieManager.
- **Step 2: Dense Semantic Embedding (384-D):** The user query is converted into a 384-dimensional dense numerical vector using sentence-transformers/all-MiniLM-L6-v2.
- **Step 3: FAISS Similarity Search (Top-K):** The query vector searches through vectorstore/db_faiss/index.faiss (119 MB) in under 20ms using Cosine/Euclidean distance, extracting the top 2 textbook passages.
- **Step 4: Augmented Prompt Construction:** The retrieved context is enclosed inside a strict system prompt instructing the LLM to ground all findings exclusively in the verified textbook literature.
- **Step 5: High-Speed LLM Inference (Groq LPUs):** Groq's Language Processing Units execute model inference (qwen/qwen3.8-27b / gpt-oss-20b) with temperature=0.3, outputting clean triage text.
- **Step 6: Voice Synthesis & UI Rendering:** extract_voice_summary creates a 2-sentence summary synthesized by gTTS into MP3 audio in 2.5s, while the card displays the verified source badge.

4. Complete File & Folder Anatomy

File / Directory	Architectural Responsibility
medibot.py	Core runtime application. Orchestrates Streamlit UI, cookie session persistence, FAISS retrieval hooks, Groq LPU API calls, translation engine, and message rendering.
create_memory_for_llm.py	Offline data pipeline. Uses PyMuPDF (fitz) to parse medical textbook PDFs, divides text with RecursiveCharacterTextSplitter (chunk=1000, overlap=100), and exports FAISS index.
vectorstore/db_faiss/	Pre-computed 119 MB knowledge store: index.faiss holds the high-dimensional index tree; index.pkl holds chunk text strings and page metadata.
style.css	Custom clinical styling. Eliminates default Streamlit UI styling in favor of modern hospital SaaS components: rounded cards, color-coded borders, responsive flexboxes.
users.json	Credential database. Stores registered usernames mapped to cryptographically salted bcrypt password hashes with 12 salt rounds.
requirements.txt	Declares dependencies: streamlit, groq, faiss-cpu, sentence-transformers, deep-translator, bcrypt, gtts, pypdf.
.env / secrets.toml	Secure storage for private GROQ_API_KEY. Excluded from version control via .gitignore.

5. Deep Dive into Core Technical Concepts

Concept 1: The 'Open-Book' RAG Model

A standard LLM behaves like a student writing an examination purely from memory; when unsure, it fabricates plausible answers. UpCharika's RAG architecture converts this into an open-book examination: it queries the FAISS index for relevant textbook passages, injects them into the context window, and forces the model to synthesize answers strictly from verified medical literature.

Concept 2: Semantic Vector Embeddings (384 Dimensions)

Computers cannot parse semantic intent from raw ASCII characters. The all-MiniLM-L6-v2 model maps sentences into a continuous 384-dimensional vector space. Phrases sharing clinical concepts (such as 'elevated body heat' and 'acute pyrexia / fever') occupy proximal coordinates, enabling accurate conceptual matching even when keywords differ.

Concept 3: Text Chunking & Chunk Overlap

Clinical textbooks contain extensive narrative chapters that exceed transformer context windows. RecursiveCharacterTextSplitter partitions text into 1,000-character segments. A 100-character overlap is preserved between successive segments to guarantee that clinical directives spanning chunk boundaries (e.g., compound drug dosages) are not severed.

Concept 4: FAISS Vector Database vs Relational SQL

Traditional relational databases (MySQL) perform exact string or index filtering (e.g. WHERE symptom='fever') and cannot compute contextual proximity. FAISS constructs an inverted file index tree enabling nearest-neighbor distance computation across thousands of vectors in sub-20ms latency.

Concept 5: Groq LPUs vs Conventional GPUs

Standard generative AI relies on GPUs (Nvidia A100/H100) which suffer from memory bandwidth bottlenecks during auto-regressive token generation. Groq's Language Processing Units (LPUs) feature high-bandwidth on-chip SRAM, yielding 250–400 tokens/sec and reducing clinical query turnaround to ~1.5 seconds.

Concept 6: Low-Latency Voice Optimization (gTTS)

Google Text-to-Speech synthesis latency scales linearly with character volume. Transcribing full 1,500-character clinical guidance initially required 82 seconds. The extract_voice_summary routine extracts the first two primary clinical triage sentences (<= 250 chars), dropping generation latency to under 3 seconds.

Concept 7: Password Security via Salted Bcrypt

User credentials stored in users.json utilize bcrypt key derivation with 12 algorithmic cost rounds. The algorithm generates a unique cryptographic salt for each entry, neutralizing rainbow-table attacks and precomputed dictionary cracking.

6. Top 10 Viva Voce & Defense Questions with Model Answers

Memorize and understand these high-frequency examiner questions for your final defense:

Q1: Why did you choose RAG over Fine-Tuning an LLM?

Answer: Fine-tuning adjusts internal neural weights, which requires extensive GPU resources, takes hours or days, risks catastrophic forgetting, and can still hallucinate. With RAG, the LLM remains frozen. We simply update the textbook PDF in our FAISS vectorstore with zero GPU re-training cost and 100% source verifiability.

Q2: What is the specific difference between your Embedding Model and the LLM?

Answer: The Embedding Model (sentence-transformers/all-MiniLM-L6-v2) is a mathematical encoder: it converts text into 384 numbers for similarity search. It cannot generate text. The LLM (Groq / Qwen / LLaMA) is a generative language model: it ingests the retrieved textbook context and writes structured clinical answers.

Q3: What mathematical metric does FAISS use to identify relevant textbook passages?

Answer: FAISS computes Cosine Similarity (the cosine of the angle between two normalized vectors) or L2 Euclidean Distance in 384-dimensional space. Smaller geometric distance signifies higher medical topic relevance.

Q4: Why is temperature set to 0.3 in your Groq inference parameters?

Answer: Temperature controls output randomness. High values (0.8–1.0) promote creative, divergent language suited for storytelling. In medical triage, hallucinations are hazardous; a low temperature of 0.3 enforces strict, deterministic, and factual responses.

Q5: How did you solve the audio lag problem in voice consultation?

Answer: By default, gTTS attempted to synthesize the entire 1,500-character diagnosis, taking 82 seconds. I engineered extract_voice_summary to isolate the first 2 primary clinical sentences (under 250 characters), reducing synthesis time down to ~2.5 seconds.

Q6: How can you prove to an examiner that an answer originated from your textbook rather than Groq's general training?

Answer: We built a Test Mode toggle in the sidebar ('Test Mode: Disable Groq'). When enabled, it bypasses Groq completely and extracts the raw textbook excerpt directly from FAISS with a distinct purple badge: '100% Direct Textbook (Groq OFF)'. When turned off, Groq refines that same excerpt with the blue badge: 'Medical Textbook Grounded + Groq AI'.

Q7: What happens if internet connectivity fails or API rate limits are exceeded?

Answer: UpCharika features a 3-tier fault tolerance fallback: Tier 1 executes True RAG (FAISS + Groq). Tier 2 falls back to Direct Textbook Extraction if Groq is unreachable. Tier 3 activates an offline Devanagari triage knowledge base for common emergencies.

Q8: How does your multilingual translation engine function?

Answer: It uses a hybrid pipeline: common emergencies trigger native Devanagari clinical triage dictionaries (zero latency); open inquiries prompt Groq to generate native Marathi/Hindi; and deep-translator (GoogleTranslator engine) dynamically re-translates conversation history upon language switching.

Q9: Why chunk overlap is set to 100 characters instead of 0?

Answer: Zero overlap can bisect critical medical guidance right down the middle across adjacent chunks (e.g. splitting a drug dosage from its contraindication). A 100-character overlap ensures contextual continuity across boundaries.

Q10: What are three immediate future directions for the UpCharika project?

Answer: 1. Integration with hospital Electronic Health Record (EHR) systems via HL7/FHIR standards. 2. Speech-to-Text input for voice-driven patient queries. 3. Computer Vision / OCR to read and interpret physical prescription slips and lab diagnostic reports.